

Coding Challenge 3: Data science IRL

Background

Data drives modern innovation in biology, but often we have to combine data collected by different people, at different times, encoded in different ways, and stored in different files. Being able to identify mistakes, reshape, recode, and combine those files is a key step. The two scenarios in this challenge are both real-world examples of the same problem, and both connect to the Sustainable Development Goals introduced in Week 1:

- **SDG 3: Good Health and Well-being** — Biological data can help to understand disease dynamics and track the spread of pathogens. Linking a patient's viral load to their microbiome and metabolome is a first step toward understanding why some patients get sicker than others.
- **SDG 6: Clean Water and Sanitation** — Biological monitoring and data analysis can help assess water quality, detect contamination, and guide conservation efforts. Arsenic in freshwater is a human-health problem as well as an ecological one.
- **SDG 14 and 15: Life Below Water and Life on Land** — Linking a pollutant to the invertebrate community and water chemistry of a lake is how biologists decide where restoration effort will matter most.

Introduction

At this point, you should be familiar with base R, `ggplot` graphics, `dplyr` for data management, and `lubridate` for working with date objects in R. Now you've learned all the basics to organize data and explore them in R! In this assignment, you'll get a chance to explore the power of these tools. To set the stage, choose one of the two scenarios below and keep it in mind as you work through the assignment:

1. **You are working as a data scientist at a hospital in the infectious disease ward.** The doctors are very skilled and knowledgeable, but they lack training in basic data science. They need you to organize and then explore their data to look for patterns that might be important for making life-saving decisions. The main issue is that the relevant data were collected by different doctors, at different times, and stored into different files. Your main job is to combine these different files into a single working data frame that you can use to generate visualizations. There are three separate datasets, but each can be linked by a patient *ID*. The *Date* column indicates the date that each patient sample was collected. The first dataset contains a single measurement *Conc* — the concentration of viral particles in the bloodstream in units called 'CT score', which is the number of PCR cycles until viral RNA or DNA is detected. The second dataset has two measurements: *PC1* and *PC2* — these are *Principal Component* scores that describe the gut microbial community. The numbers can't be interpreted directly, but the important thing is that patients with similar *PC1* and *PC2* scores have similar gut communities, while patients with different scores have different communities. In other words, the difference in *PC1* and *PC2* is a quantitative measure of how different these communities are. The third dataset also has *PC1* and *PC2*, but these are urine metabolites (i.e. small molecules like glucose and hormones), also known as the urine metabolome. Similar to the gut microbial community, *PC1* and *PC2* of the urine metabolome can't be interpreted alone, but can help to compare how similar the patient urine samples are in terms of their metabolome. *Your main goal is to test the hypothesis that patients with similar viral loads have similar gut microbes and urine metabolites. A secondary goal is to see if viral load differs by year.*
2. **You are working as a data scientist for an NGO that is dedicated to habitat restoration.** The field ecologists are very skilled and knowledgeable, but they lack training in basic data science. They need you to organize and then explore their data to look for patterns that might be important for making conservation decisions. The main issue is that the relevant data were collected by different ecologists, at different times, and stored into different files. Your main job is to combine these different

files into a single working data frame that you can use to generate visualizations. There are three separate datasets, but each can be linked by an *ID* code that is unique to a specific lake within the Laurentian Great Lakes watershed. The *Date* column indicates the date that a water sample was taken from each lake. The first dataset includes a single measurement *Conc* — the concentration of arsenic in the water, in units of microgram per litre (g/L). The second dataset has two measurements: *PC1* and *PC2* — these are *Principal Component* scores that describe the invertebrate community. The numbers can't be interpreted directly, but the important thing is that lakes with similar PC1 and PC2 scores have similar invertebrate communities, while lakes with different scores have different communities. In other words, the difference in PC1 and PC2 is a quantitative measure of how different these communities are. The third dataset also has *PC1* and *PC2*, but these are calculated from water chemistry measurements (e.g. dissolved oxygen, turbidity). Similar to the invertebrate community, PC1 and PC2 of the water chemistry can't be interpreted alone, but can help to compare how similar the lake samples are in terms of their chemistry. *Your main goal is to test the hypothesis that lakes with similar arsenic levels have similar invertebrate communities and water chemistry. A secondary goal is to see if arsenic concentration differs by year.*

In either scenario, you'll have to start by working with the different data files. This is an example of **relational data** because the different data files are *related* via a specific ID code. The kind of code you will write is a simplified version of what you might get from a large relational database (some popular examples are SQL and mongoDB).

You'll need to combine the individual files into a larger `data.frame` or `tibble` object, run some basic quality controls to look for typos and missing data, and then reorganize the data for plotting. The .Rmd template names the section beside each task.

Using the template

1. **Get organized.** Create a new project folder for this challenge (New → New Folder from Template → R Project), and add a `data` folder inside it.
2. **Download the assignment files from OnQ:** the template `BIOL343-CC3-template.Rmd` (save it in the project folder and **rename it**, e.g. `Group_#_CC3.Rmd`) and the three data files `Dataset1.csv`, `Dataset2.csv`, `Dataset3.csv` (save them in `data`). Keep the data file names exactly as they are, and **do not edit the data files by hand**: every correction is done in R so that it is documented.
3. **Open your project folder in Positron** (File → Open Folder...) so that relative paths like `data/Dataset1.csv` work.
4. **Start your coding log** by following the **DO THIS FIRST** block at the top of the template, before you write any code.
5. **Choose your scenario** and write it on the line under *Group Members*. The code is the same for both; only the column names you choose, the labels and the interpretation differ.
6. **Work through the numbered sections.** Each one has a short instruction, the Chapter 10 section it comes from, and an empty code chunk. Write your code in the chunk, run it, and read the output before moving on. Where the template asks a **Question**, type your answer on the **Answer:** line as ordinary text, outside the code chunk.
7. **Knit as you go.** Run `rmarkdown::render("Group_#_CC3.Rmd")` in the Console (with your file's name). Fix any error before adding more code. Before you upload, open the `.html` in a web browser and check that the figures are visible: the **Render** button in Positron can write figures to a separate folder, in which case the uploaded file shows a broken image.
8. **Finish as a group.** Compare your individual versions, agree on one, knit it, and upload **both** the `.Rmd` and the `.html`, named `Group_#_CC3.Rmd` and `Group_#_CC3.html` (your group number in place of #).

Two things to expect. `Dataset3.csv` was exported by a collaborator whose software separates columns with a **semicolon** instead of a comma; the help pages for `read.csv()` and `read_delim()` show the argument

that handles this. And two of the files record missing measurements with a **word** instead of leaving the cell blank, which makes R read the whole column as text: `str()` will show you, and the template walks you through the fix.

Your coding log (positpal)

The routine is the same as the earlier challenges, and the commands are in the template under **DO THIS FIRST** and **DO THIS LAST**. Copy and paste them into the R Console; they are not code chunks and must not be added as code chunks.

With your project folder open in Positron (so that it is your working directory) and the template saved there under the name you will upload, start your log **before you write any code**, replacing `FILENAME.Rmd` with that file name:

```
install.packages("remotes")
remotes::install_url("https://quantitative.bio/downloads/positpal.tar.gz", upgrade = "never")
positpal::start("FILENAME.Rmd")
```

Check that autosave is on in Positron (*File* → *Auto Save*). The first time you use positpal on a computer, `start()` will ask you to run `positpal::accept_eula()` once; do that, then run the `start()` line again. Run the `start()` line again every time you sit down to work on the file, and check after a few minutes that the `.capture/snaps` folder in your project is filling up. If `start()` refuses to record, run `positpal::doctor("FILENAME.Rmd")` and bring the output to tutorial or office hours. positpal records every file in the project folder, so keep only this challenge's files in it.

When you are finished (after the group's final knit), each group member runs:

```
positpal::stop_capture()
positpal::export_capture()
```

`export_capture()` prints the name and location of the `my-capture-YYYYMMDD-HHMMSS.zip` it creates beside your `.Rmd`; that is your coding log. Upload it to **CC3 — Coding Log (individual)**. If `export_capture()` fails, zip the `.capture` folder yourself and upload that instead.

Stuck on an error? Run `positpal::coach_brief()` and paste the text it prints into the course Coding Coach (OnQ → Course Help) along with your questions. If that doesn't help you address the problem, then email the entire Coding Coach conversation to the course instructor.

Submission

- **One per group:** upload `Group_#_CC3.Rmd` and `Group_#_CC3.html` to your section's folder — **TUT02-CC3** (section 002) or **TUT03-CC3** (section 003). Submitting to the wrong section's folder, or with a group you were not assigned to, carries the penalty described in the syllabus.
- **One per person:** upload your `my-capture-....zip` to **CC3 — Coding Log (individual)**. Upload the `.zip`, not your `.Rmd`.
- Coding challenges are graded 0–5 for completeness.

Friendly reminder: to avoid losing marks, be sure to include the full names and student IDs of each group member at the top of your submitted document. You may complete this assignment individually if you have accommodations or extenuating circumstances.